

Making a Bot Behave

Adaptive Forms | Buttons | Clarification | Human Handover

Session 8 | BSBI Berlin | MSc Artificial Intelligence

Contents

If this page is blank, press Ctrl+A then F9.

Contents

2

If this page is blank, press Ctrl+A then F9.

2

1 What we are adding today

12

Your bot can already understand messages and pick responses. What it cannot yet do is hold a proper conversation - one where it asks follow-up questions, adapts to your answers, recovers when it misunderstands, and knows when to give up and fetch a human.

12

Four things, and they connect into one flow:

12

"which tool should I use?"

12

|

12

FORM starts <---- BUTTONS make answers reliable

12

|

12

VALIDATION <---- rejects nonsense, asks again

12

|

12

ADAPTIVE QUESTIONS <-- the form changes based on answers

12

|

12

recommendation

12

...but if the bot gets lost at any point:

12

|

12

TWO-STAGE CLARIFICATION

12

|
12

still lost?
12

|
12

HUMAN HANDOVER
12

Every one of these is named in your assignment brief. This is not extra material - it is the part of the specification you have not been shown yet.
13

2 Forms that adapt
14

2.1 The problem with the form you already have
14

Your bot has `tool_comparison_form`. It asks two questions, in the same order, every time. Watch what happens when someone does not co-operate:
14

Nothing checked the answers. Nothing adapted. It is a paper form with a chat interface painted on top.
14

2.2 The same conversation, done properly
14

Now the new form. Read it twice - pay attention to the third question.
14

2.3 How it works: `required_slots()`
17

One method does all of it. Rasa calls it after every answer to ask: what do you still need?
17

```
async def required_slots(self, domain_slots, dispatcher, tracker, domain):  
17
```

```
    slots = list(domain_slots)          # ["build_type", "hosting"]  
17
```

```

    if tracker.get_slot("hosting") == "local":
17
        slots.append("experience")    # add a question
17
    elif tracker.get_slot("hosting") == "cloud":
17
        slots.append("budget")        # add a different one
17
    return slots
17

```

domain_slots is what you declared in domain.yml - the questions that are always asked. You return that list plus whatever else the situation calls for. The form then works through your list.

17

Because Rasa asks again after every answer, the form can keep changing its mind as the conversation develops.

17

2.4 How it works: validate_<slot_name>()

17

One method per slot you want to check. The name matters - Rasa finds it by matching validate_ plus the slot name.

17

```

def validate_budget(self, slot_value, dispatcher, tracker, domain):
17

```

17

```

    digits = "".join(ch for ch in str(slot_value) if ch.isdigit())
17

```

17

```

    if not digits:
17

```

17

```

        dispatcher.utter_message(text="I need a number...")
17

```

17

```

    return {"budget": None}    # None -> REJECT, ask again
18

```

18

```

    return {"budget": digits}    # value -> ACCEPT, move on
18

```

18

That return value is the entire mechanism. There is no loop for you to write - the form does the looping. People find this hard to believe at first.

18

Why does budget need this and the others do not? Because budget is free text. The button-driven slots can only produce values you chose. Free text always needs validation; buttons mostly do not. That is an argument for buttons.

18

3 Buttons

19

3.1 You have not done this before

19

You will think you have. In your Streamlit app you wrote:

19

```
if st.button("What is Rasa?"):
```

19

```
    user_input = "what is Rasa?"
```

19

Those buttons live in the user interface. They are hardcoded by you, and the bot has no idea they exist. They just type a sentence on your behalf, which then goes through NLU like anything else.

19

Your assignment brief asks for something different:

19

3.2 Buttons the bot sends

19

In domain.yml:

19

```
utter_ask_hosting:
```

19

```
- text: "Where do you want it to run?"
```

19

```
buttons:
```

19

```
- title: "On my own machine"
```

19

```
    payload: '/inform_hosting{"hosting":"local"}'
```

19

```
- title: "In the cloud"
19
payload: '/inform_hosting{"hosting":"cloud"}'
```

Or generated in Python, which is what the brief actually asks for:

```
19 dispatcher.utter_message(
19     text="Which destination?",
19     buttons=[{"title": city,
19               "payload": f'/inform_destination{{"city":"{city}"}}'}]
19     for city in top_destinations]
20 )
20
```

3.3 The payload is the important part

```
20 /inform_hosting{"hosting":"cloud"}
20
```

This is not a sentence. It is an instruction. It tells Rasa directly:

3.4 Your Streamlit app currently throws buttons away

Look at what your UI does with the bot's reply:

```
21 reply = rasa_replies[0]["text"]      # the "buttons" key is ignored
21
```

But the bot actually sends this:

```
21 [{
21     "recipient_id": "user1",
21
```

```

    "text": "Where do you want it to run?",
21
    "buttons": [
21
        {"title": "In the cloud",
21
         "payload": "/inform_hosting{\"hosting\":\"cloud\"}"}
21
        ]
21
    ]}]
21

```

The buttons are right there, unused. Rendering them is a small change:

```

21
    for msg in rasa_replies:
21
        if msg.get("text"):
21
            st.write(msg["text"])
21
        for b in msg.get("buttons", []):
21
            if st.button(b["title"], key=b["payload"]):
21
                send_to_rasa(b["payload"])    # PAYLOAD, not title
21

```

4 Two-stage clarification

4.1 What your bot does now

One misunderstanding and the user is stuck. They have to work out for themselves how to rephrase, with no clue what the bot does understand.

4.2 What it should do

4.3 Rasa already has this built in

24

You do not have to write the loop. It is called `action_two_stage_fallback`, and you switch it on with one rule:

24

- rule: two stage fallback

24

- steps:

24

- intent: `nlu_fallback`

24

- action: `action_two_stage_fallback`

24

- active_loop: `action_two_stage_fallback`

24

Notice `active_loop`. That is there because the two-stage fallback is a loop - exactly the same machinery as a form. It keeps control of the conversation until it either succeeds or gives up.

24

4.4 Where the button options come from

24

This is the satisfying part. The clarification options are not hardcoded:

24

```
ranking = tracker.latest_message.get("intent_ranking", [])
```

24

```
candidates = [r for r in ranking
```

24

```
if r.get("name") != "nlu_fallback"][:2]
```

24

The default version of this action shows the raw intent name - "Did you mean 'ask_what_is_nlu'?" - which means nothing to a user. Rasa's own documentation recommends overriding it, which is why we map intent names to readable labels.

25

5 Human handover

26

5.1 Handing over is not failing

26

Your brief requires it explicitly:

26

A bot that escalates cleanly is more trustworthy than one that keeps guessing. The skill being assessed is the context packaging - a human picking this up must not have to ask the user to start again.

26

5.2 Reading the whole conversation

26

You already know `tracker.get_slot()`. That gives you individual values. To hand over properly you need the conversation itself:

26

```
transcript = []
```

26

```
for event in tracker.events:
```

26

```
    if event.get("event") == "user":
```

26

```
        transcript.append(f"USER: {event.get('text')}")
```

26

```
    elif event.get("event") == "bot":
```

26

```
        transcript.append(f"BOT : {event.get('text')}")
```

26

5.3 What a useful package contains

26

```
handover_package = {
```

26

```
    "conversation_id": tracker.sender_id,
```

27

```
    "collected":    slots,          # everything gathered so far
```

27

```
    "last_intent":   last_intent,    # what it thought you meant
```

27

```
    "last_confidence": confidence,   # how sure it was
```

27

```
"turn_count": len(transcript),  
27  
"transcript": transcript[-20:],  
27  
}  
27
```

Ask yourself what a human advisor actually needs in order to take over without irritating the customer. That list is your design decision, and it is assessable.

27

6 Reference

28

6.1 Which file does what

28

6.2 Write one story per branch

28

Your form has two paths - local and cloud. Write a story for each. If you only write one, TEDPolicy only ever learns that path.

28

6.3 Mistakes to avoid

28

6.4 Before you train

28

```
rasa data validate    # catches missing intents and actions in  
seconds
```

28

```
rasa train
```

28

Run rasa data validate first, always. It finds intents used in stories but missing from the domain, undefined actions, and slot mapping mistakes - before you spend fifteen minutes training.

28

Making a Bot Behave | Advanced Conversational UI Design &
Chatbot Development | BSBI Berlin

28

1 What we are adding today

Your bot can already understand messages and pick responses. What it cannot yet do is hold a proper conversation - one where it asks follow-up questions, adapts to your answers, recovers when it misunderstands, and knows when to give up and fetch a human.

Four things, and they connect into one flow:

"which tool should I use?"

|

FORM starts <---- BUTTONS make answers reliable

|

VALIDATION <---- rejects nonsense, asks again

|

ADAPTIVE QUESTIONS <-- the form changes based on answers

|

recommendation

...but if the bot gets lost at any point:

|

TWO-STAGE CLARIFICATION

|

still lost?

|

HUMAN HANDOVER

Every one of these is named in your assignment brief. This is not extra material - it is the part of the specification you have not been shown yet.

What the brief asks for	What we build today
"Trip intake module with adaptive questioning "	<code>required_slots()</code>
" Error recovery for incomplete or ambiguous messages"	<code>validate_<slot>()</code>
"quick-reply buttons generated from custom action responses "	Buttons in responses and actions
" two-stage clarification flow ... before escalating"	<code>action_two_stage_fallback</code>
" Human advisor escalation with full context handover"	<code>action_handover_to_human</code>

2 Forms that adapt

2.1 The problem with the form you already have

Your bot has `tool_comparison_form`. It asks two questions, in the same order, every time. Watch what happens when someone does not co-operate:

Your current form
Bot: Which is the first tool you want to compare?
You: banana
Bot: And the second tool?
You: also banana
Bot: Here is your comparison:
Banana: I don't have detailed notes on this one yet.
Banana: I don't have detailed notes on this one yet.

Nothing checked the answers. Nothing adapted. It is a paper form with a chat interface painted on top.

2.2 The same conversation, done properly

Now the new form. Read it twice - pay attention to the third question.

Answering LOCAL

You: which tool should I use?

Bot: What are you planning to build?

[A chatbot] [A voice assistant] [An FAQ bot]

You: *clicks A chatbot*

Bot: Where do you want it to run?

[On my own machine] [In the cloud]

You: *clicks On my own machine*

Bot: How much Python experience do you have? <-- THIS

[I am a beginner] [I am comfortable with Python]

You: *clicks I am a beginner*

Bot: For a chatbot running local, I would suggest Botpress -
a visual builder, so you need less Python.

Answering CLOUD instead

You: which tool should I use?

Bot: What are you planning to build?

[A chatbot] [A voice assistant] [An FAQ bot]

You: *clicks An FAQ bot*

Bot: Where do you want it to run?

[On my own machine] [In the cloud]

You: *clicks In the cloud*

Bot: Roughly what monthly budget do you have? <-- DIFFERENT

The point

Same form. Different third question. The form decided what to ask next based on what you had already told it.

That is what your assignment brief means by **adaptive questioning**. Without it you have a questionnaire, not

2.3 How it works: required_slots()

One method does all of it. Rasa calls it after every answer to ask: what do you still need?

```
async def required_slots(self, domain_slots, dispatcher, tracker, domain):
```

```
    slots = list(domain_slots)          # ["build_type", "hosting"]
```

```
    if tracker.get_slot("hosting") == "local":
```

```
        slots.append("experience")      # add a question
```

```
    elif tracker.get_slot("hosting") == "cloud":
```

```
        slots.append("budget")          # add a different one
```

```
    return slots
```

`domain_slots` is what you declared in `domain.yml` - the questions that are always asked. You return that list plus whatever else the situation calls for. The form then works through your list.

Because Rasa asks again after every answer, the form can keep changing its mind as the conversation develops.

A small trap

Write `list(domain_slots)`, not `domain_slots`. If you modify the list you were given, you change it for every future turn as well. The bug appears intermittently, which makes it horrible to find.

2.4 How it works: validate_<slot_name>()

One method per slot you want to check. The name matters - Rasa finds it by matching `validate_` plus the slot name.

```
def validate_budget(self, slot_value, dispatcher, tracker, domain):
```

```
    digits = "".join(ch for ch in str(slot_value) if ch.isdigit())
```

```
    if not digits:
```

```
        dispatcher.utter_message(text="I need a number...")
```

```
return {"budget": None}          # None -> REJECT, ask again
```

```
return {"budget": digits}        # value -> ACCEPT, move on
```

You return	What happens
{"budget": None}	Rejected. The form asks the same question again.
{"budget": "40"}	Accepted. The form moves to the next slot.

That return value is the entire mechanism. There is no loop for you to write - the form does the looping. People find this hard to believe at first.

Why does budget need this and the others do not? Because budget is free text. The button-driven slots can only produce values you chose. Free text always needs validation; buttons mostly do not. That is an argument for buttons.

3 Buttons

3.1 You have not done this before

You will think you have. In your Streamlit app you wrote:

```
if st.button("What is Rasa?"):

    user_input = "what is Rasa?"
```

Those buttons live in the user interface. They are hardcoded by you, and the bot has no idea they exist. They just type a sentence on your behalf, which then goes through NLU like anything else.

Your assignment brief asks for something different:

From the brief

"quick-reply buttons **generated dynamically from custom action responses** for destination and preference selection"

The bot must send the buttons. Not the UI.

3.2 Buttons the bot sends

In `domain.yml`:

`utter_ask_hosting`:

```
- text: "Where do you want it to run?"

  buttons:

    - title: "On my own machine"

      payload: '/inform_hosting{"hosting":"local"}'

    - title: "In the cloud"

      payload: '/inform_hosting{"hosting":"cloud"}'
```

Or generated in Python, which is what the brief actually asks for:

```
dispatcher.utter_message(

    text="Which destination?",

    buttons=[{"title": city,

               "payload": f'/inform_destination>{"city": "{city}"'}
```

```
        for city in top_destinations]

    )
```

3.3 The payload is the important part

```
/inform_hosting{"hosting": "cloud"}
```

This is not a sentence. It is an instruction. It tells Rasa directly:

- the intent is `inform_hosting`
- the entity `hosting` has the value `cloud`

NLU is skipped entirely. No classification, no confidence score, no chance of a fallback. It is the most reliable input your bot will ever receive.

That is why buttons are the standard answer to "the bot keeps misunderstanding this question".

So why still write training examples for `inform_hosting`?

Because people type instead of clicking. Someone will answer "in the cloud" rather than pressing the button, and the form must still work.

Buttons are a convenience, not a guarantee. A form that only works if you click is a fragile form - and your brief asks for "error recovery mechanisms that handle incomplete or ambiguous messages".

3.4 Your Streamlit app currently throws buttons away

Look at what your UI does with the bot's reply:

```
reply = rasa_replies[0]["text"]          # the "buttons" key is ignored
```

But the bot actually sends this:

```
[{
    "recipient_id": "user1",
    "text": "Where do you want it to run?",
    "buttons": [
        {"title": "In the cloud",
         "payload": "/inform_hosting{\"hosting\": \"cloud\"}"}
    ]
}]
```

The buttons are right there, unused. Rendering them is a small change:

```
for msg in rasa_replies:
    if msg.get("text"):
        st.write(msg["text"])

    for b in msg.get("buttons", []):
        if st.button(b["title"], key=b["payload"]):
            send_to_rasa(b["payload"])    # PAYLOAD, not title
```

The bug everyone writes at least once

Sending `b["title"]` instead of `b["payload"]`.

It works - for a while. "In the cloud" happens to classify correctly, so the button appears to function. Then one day a button title does not classify, and the bot misunderstands a click. Send the payload.

4 Two-stage clarification

4.1 What your bot does now

One strike and out

You: wht is nlu thing

Bot: I'm sorry, I didn't quite understand that.

...and that is the end of the conversation.

One misunderstanding and the user is stuck. They have to work out for themselves how to rephrase, with no clue what the bot does understand.

4.2 What it should do

Recovering from a misunderstanding

You: wht is nlu thing

Bot: Sorry, I'm not sure what you meant. Were you asking about:

[what NLU means] [what Rasa is] [None of these]

<-- STAGE 1

You: *clicks what NLU means*

Bot: NLU stands for Natural Language Understanding...

...conversation recovered.

When stage 1 does not work

Bot: Sorry, I'm not sure what you meant. Were you asking about:

[what NLU means] [what Rasa is] [None of these]

You: *clicks None of these*

Bot: Sorry, I still did not follow. Could you say that
a different way? <-- STAGE 2

You: no just get me a person

Bot: I have passed your details to a human advisor...
<-- GIVE UP

4.3 Rasa already has this built in

You do not have to write the loop. It is called `action_two_stage_fallback`, and you switch it on with one rule:

```
- rule: two stage fallback

  steps:

    - intent: nlu_fallback

    - action: action_two_stage_fallback

    - active_loop: action_two_stage_fallback
```

Notice `active_loop`. That is there because the two-stage fallback is a loop - exactly the same machinery as a form. It keeps control of the conversation until it either succeeds or gives up.

Stage	Action that runs	What the user sees
trigger	nlu_fallback detected	-
1	action_default_ask_affirmation	"Did you mean X?" with buttons
success	-	fallback undone, message replayed
2	action_default_ask_rephrase	"Say it differently"
give up	action_default_fallback	override this to hand over

Two things that will catch you out

1. You must define `utter_ask_rephrase` in your domain. If you do not, stage two does **nothing at all** - no error, no message, complete silence. Rasa builds that action with `silent_fail=True`.

2. Remove your old `Handle NLU fallback` rule. It sends `nlu_fallback` straight to `utter_default`, and it will fight the new rule for the same intent.

4.4 Where the button options come from

This is the satisfying part. The clarification options are not hardcoded:

```
ranking = tracker.latest_message.get("intent_ranking", [])

candidates = [r for r in ranking

               if r.get("name") != "nlu_fallback"][:2]
```


You have seen `intent_ranking` before

It is the ranked list DIET produces for every message - the one we pulled out of `conversations.db` inside `parse_data` when we studied `config.yml`.

At the time it looked like trivia. It is now choosing what the bot offers you when it is confused. Nothing in that pipeline was decoration.

The default version of this action shows the raw intent name - "Did you mean 'ask_what_is_nlu'?" - which means nothing to a user. Rasa's own documentation recommends overriding it, which is why we map intent names to readable labels.

5 Human handover

5.1 Handing over is not failing

Your brief requires it explicitly:

From the brief

"Escalate complex planning cases to human travel advisors, **supplying them with full conversational context.**"

A bot that escalates cleanly is more trustworthy than one that keeps guessing. The skill being assessed is the context packaging - a human picking this up must not have to ask the user to start again.

5.2 Reading the whole conversation

You already know `tracker.get_slot()`. That gives you individual values. To hand over properly you need the conversation itself:

```
transcript = []

for event in tracker.events:

    if event.get("event") == "user":

        transcript.append(f"USER: {event.get('text')}")

    elif event.get("event") == "bot":

        transcript.append(f"BOT : {event.get('text')}")
```

You have seen this data before too

`tracker.events` is the same event stream you queried with SQL in the analytics session -
`type_name='user'` and `type_name='bot'` in `conversations.db`.

Same data. Reached from inside a custom action instead of from a database.

5.3 What a useful package contains

```
handover_package = {
```

```
"conversation_id": tracker.sender_id,  
  
"collected":      slots,          # everything gathered so far  
  
"last_intent":     last_intent,    # what it thought you meant  
  
"last_confidence": confidence,     # how sure it was  
  
"turn_count":      len(transcript),  
  
"transcript":      transcript[-20:],  
  
}
```

Ask yourself what a human advisor actually needs in order to take over without irritating the customer. That list is your design decision, and it is assessable.

Be honest in your report

The demo prints this package to the console. A real system would send it to a ticketing system, a Slack webhook, or an email.

Describing an integration you did not build is perfectly legitimate. Implying you built it is not.

6 Reference

6.1 Which file does what

File	What goes in it
domain.yml	Slots, entities, responses with buttons, the form, action names
data/nlu.yml	Training examples for the new intents, plus synonyms
data/rules.yml	Form activation, form submission, two-stage fallback, escalation
data/stories.yml	Multi-turn journeys - one per adaptive branch
actions/actions.py	FormValidationAction, handover, custom affirmation

6.2 Write one story per branch

Your form has two paths - local and cloud. Write a story for each. If you only write one, **TEDPolicy** only ever learns that path.

6.3 Mistakes to avoid

Mistake	What you will see
Old <code>Handle NLU fallback</code> rule left in	Two-stage fallback never runs
<code>utter_ask_rephrase</code> not defined	Stage two is completely silent
Sending button title instead of payload	Works sometimes, fails oddly later
Slots not cleared after the form	Second run skips questions
Class not named <code>validate_<form_name></code>	Validation silently never runs
Modifying <code>domain_slots</code> in place	Wrong questions, intermittently
Dynamic slot has no mapping in the domain	Form asks the same thing forever

6.4 Before you train

```
rasa data validate          # catches missing intents and actions in seconds
```

```
rasa train
```

Run `rasa data validate` first, always. It finds intents used in stories but missing from the domain, undefined actions, and slot mapping mistakes - before you spend fifteen minutes training.